Current model update: fine-coordinate retrieval

Revised 21 October 2026. Read this before older L5-L7 experiment descriptions. The current Kerala demo defaults to L10 when its assets are installed.

What changed

The original classifier chooses one of 300 geographic cells and returns the stored cell center. L7 adds training-neighbor class votes but retains those same 300 output coordinates. L10 instead matches the uploaded photograph against 15,939 training references and returns the matched photo's GPS coordinate. Thus there are 15,939 reference positions before duplicate coordinates, rather than 300 centers. This is still reference-based localization, not unrestricted latitude-longitude regression.

A ResNet-50 encoder produces 2,048 numbers and a frozen DINOv2 ViT-S/14 encoder produces 384. Each vector is L2-normalized, scaled by the square root of 0.5 and concatenated. A cosine search chooses the most similar reference. The resulting score is an appearance similarity, not a calibrated probability. The displayed reference is from training; the selected demo query is from the official test set.

Measured comparison

Same 357 official test images; the benchmark was reused across experiments. These are exploratory follow-ups, not independent confirmation.

Method	Median error	Within 5 km	Within 25 km
L5 classifier	42.08 km	historical baseline	41.18%
L6 regularized head	28.41 km	see experiment record	47.62%
L7 classifier + KNN votes	21.84 km	8.40%	53.50%
L8 ResNet local coordinate retrieval	18.27 km	15.13%	59.66%
L10 ResNet + DINO reference retrieval	15.92 km	17.37%	63.03%

The L10 result is 225/357 within 25 km and 62/357 within 5 km. None is within 1 km: every official test photo is over 1 km from the original training reference set. On random validation, L10 is 71.1% within 5 km and median 1.07 km; that split contains many nearby locations and must not be presented as unseen-place performance.

What to explain and what remains limited

Why two encoders?

An encoder transforms image pixels into a feature vector, or descriptor. ResNet was trained for our geographic classification task; DINOv2 provides a different, self-supervised visual representation. Their normalized cosine similarities contribute equally. DINO alone did not beat the L8 validation baseline, while the combined descriptor did. Complementary information is a plausible explanation; our small experiment does not prove the exact mechanism.

How was the winner chosen?

L8 evaluated four fixed coordinate-retrieval recipes on validation. L9 tested DINO-only retrieval and failed its promotion gate, so it was not tested on the official test set. L10 evaluated six fixed mixtures/neighborhood choices, prioritizing validation within 5 km with a guardrail on within 25 km. The equal mixture with one neighbor won. Test results were computed after this selection. All encoders were frozen during these retrieval comparisons; this was not an end-to-end retraining run.

Is this ETH Zurich's method?

No. ETH/Google's Scaling Image Geo-Localization to Continent Level combines learned ground prototypes with aerial embeddings and fine S2 cell codes. Its ground encoder searches that fused geographic representation. Our deployment searches ground photographs with two encoders; it has no aerial encoder, learned aerial-ground fusion or comparable training scale. ETH's reported within-200-m Europe result is not comparable to our within-25-km Kerala-area result. We have not surpassed or reproduced that system.

Why can a foreign photo still get a Kerala-area answer?

The regional pipeline always returns its best available match. It cannot establish that an input belongs to Kerala or is represented in the bank. The collection uses a bounding rectangle that also includes parts of neighboring states. A Great Wall image may therefore get an Indian match. A large similarity does not certify location. Neither restricting the map nor adding KNN solves this open-set recognition problem.

New data and storage

Panoramax yielded 936 candidates, 789 unique images after exact-image deduplication. These were encoded in small batches with raw downloads kept only in memory. Four review sheets and compact descriptors were retained. The first 144 images were inspected: menus, interiors and repeated frames were present. A reviewed, spatially filtered and thinned 24-reference subset did not improve validation, so it was not added to deployment. More images are not automatically better data.

Current limitations

Sparse references, viewpoint/weather changes, similar scenery, imperfect coordinates, nearby-frame leakage and regional assumptions all matter. Meter-level localization usually needs much denser coverage, appropriate aerial imagery and/or local geometric verification. The present median is roughly 16 km, not street-address accuracy.

Historical foundation and experiments

The following chapters explain the original classifier and L6/L7 experiments. References to the final L7 checkbox or 300-center output describe that historical mode. Use the preceding update and current Demo Playbook for L10.

KPlaceNet

Understand the project, then explain it

A learning guide for the final Kerala-focused demonstration

What this guide teaches: how a photograph becomes a location estimate; how the model learns; why ResNet, geographic cells and KNN are used; what the results mean; and where the system fails.

Starting knowledge: no deep-learning background is assumed. Terms are introduced before they are used. Equations are followed by plain-language explanations and small examples.

Current system: the Kerala-area demo uses the L6 ResNet classifier, optionally refined by L7 KNN. The checkbox enables this refinement when the matching training bank is installed. World mode is a separate model, not a wider search performed by the Kerala model.

How to study

Read Chapters 1-10 to understand the data, model and two improvement stages. Read Chapters 11-14 to interpret the evidence and limitations. Chapters 15-17 cover the secondary global work, research context and code. Finish with the practice questions and answer key. The optional reference pages explain the historical global experiments and summarize the equations.

After each chapter, cover the text and answer its checkpoint aloud. If an answer is difficult, return to the explanation rather than memorizing a sentence. Use the separate Demo Playbook only after you can trace the prediction pipeline yourself.

Learning map

Chapters	Question you will be able to answer
1-3	What is the task, what data is used, and why turn geography into classes?
4-6	What happens to the pixels, and what does ResNet learn?
7-10	What changed in L5, L6 and L7, and how does the final hybrid work?
11-14	How good is it, how trustworthy is the score, and what can go wrong?
15-17	What belongs to World mode, the papers and your contribution?
18-19	Can you explain the project without reading this guide?

Important distinction: an explanation marked *worked example* is invented to teach the calculation. A table marked *measured result* comes from the saved project records. Do not present a teaching example as experimental evidence.

The final regional benchmark contains 357 official test photographs. That sample size, the training assumptions and the distinction between L6 and L7 matter more than memorizing many percentages.

1. What problem are we solving?

Geolocation from visual evidence

Geolocation means estimating where something is on Earth. Here the input is one photograph and the output is a geographic estimate expressed as latitude and longitude.

Latitude measures north-south position in degrees, from -90 to +90. **Longitude** measures east-west position, from -180 to +180. A pair such as 9.6761, 76.4041 is a coordinate, not a city name or an accuracy score.

The model receives image pixels. It does not use the uploaded photograph's GPS or EXIF metadata, a filename, a gallery caption, Google reverse-image search or a live web search to identify the location. Stored training coordinates are used to teach the model and label its neighbor bank. These are different roles for coordinates.

AI, machine learning and deep learning

Artificial intelligence (AI) is the broad field of building systems that perform tasks associated with intelligence. **Machine learning (ML)** is an approach in which behavior is learned from examples rather than fully specified as rules. **Deep learning** uses neural networks with many layers to learn representations of the input.

KPlaceNet is **supervised learning**: its training examples pair photographs with known coordinates. Those coordinates are converted into location-class labels. The model learns from these labeled examples; it is not discovering locations without supervision.

Classification versus coordinate regression

Classification selects one of a fixed set of labels. **Regression** predicts numeric values directly. KPlaceNet uses classification: it selects one of 300 geographic cells and returns that cell's saved center. It does not directly regress an unrestricted latitude-longitude pair.

This simplifies the learning target, but limits precision. Several different photographs assigned to the same predicted class receive exactly the same coordinate. A displayed coordinate with four decimal places can still be tens of kilometers from the real camera location.

A clear project definition

KPlaceNet estimates a photograph's geographic region using a ResNet-50 image representation, a classifier over geographic cells, and an optional training-image retrieval refinement. The interface displays the winning cell's representative coordinate.

Checkpoint: Why is the output a regional estimate even though the website displays precise-looking coordinates? Answer key: A1.

2. The data and the regional assumption

Dataset, sample, label and metadata

A **dataset** is a collection of examples. A **sample** is one example, such as one photograph. A **label** is the desired answer used for supervised learning. **Metadata** describes a sample: its identifier, image path, coordinates or source.

The project uses subsets of **OSV-5M**, a street-view dataset containing photographs and geographic labels. A **street-view image** depicts a scene from near ground level, often a road. An aerial photograph or an indoor photograph has a different appearance distribution.

For the regional pipeline, an image's labeled coordinate is converted into a cell ID. Thus the source label is a coordinate, but the classifier's training label is a class number. The CSV connects each image path with its coordinate and identity; it is not the input tensor consumed by ResNet.

The regional data split

Partition	Images	Role
Training	15,939	Learn weights and populate the final KNN bank
Random validation	1,000	Select the L6 recipe/epoch and gate the fixed L7 refinement
Official test	357	Measure location errors after selection

The 16,939 source-training images were divided using a reproducible random seed of 42. A **random seed** initializes a pseudorandom generator so a procedure can be repeated. It does not make the resulting split geographically independent or make the model inherently accurate.

Why it is called Kerala-area mode

The extraction filter is a **bounding box**: latitude 8.0-12.8 and longitude 74.5-77.5. A bounding box is a rectangle specified by coordinate limits. It is not Kerala's administrative outline; the rectangle includes neighboring territory, including parts of Karnataka.

Selecting this mode tells the system to use that regional task. It does not first verify that the photograph was taken in Kerala. A photograph from China still receives regional class scores. A predicted point outside Kerala's administrative boundary is possible because the training geography itself is not restricted by a state polygon.

Regional specialization is useful because the model can concentrate its classes and examples on a smaller area. It also narrows the assumption under which the result is meaningful. The regional and World models have different weights and geographic class maps.

Checkpoint: Does selecting Kerala mode prove that the uploaded image is from Kerala? Answer key: A2.

3. Turning geography into classes

Cell, cell ID, centroid and bounds

A **geographic cell** is one class representing a group of training locations. Its **cell ID** is an integer identifying that class. The ID has no natural geographic ordering: class 196 is not necessarily next to class 197.

A **centroid** is a representative center of a group. Each saved cell stores centroid latitude and longitude. The predicted cell's centroid becomes the displayed coordinate. The cell's **bounds** describe its stored latitude-longitude extent; the rectangle drawn on the map is not a statistical confidence interval.

What K-means does

K-means is a clustering algorithm. It groups points around K centers by alternating between assigning points to centers and updating those centers. Here its input is training GPS coordinates, not image pixels or ResNet features.

The regional pipeline creates 300 coordinate clusters using training data only. It scales longitude by the cosine of mean training latitude to approximate local distance geometry. This is an approximation, not a full equal-area map projection. Saved cell definitions are reused by L5, L6 and L7.

When assigning labels, the implementation checks cell bounds, resolves overlapping candidates using distance to their centers, and uses a nearest-center fallback when no bounds contain the coordinate. Cluster bounding rectangles can overlap; they are not administrative districts or a perfect non-overlapping map tiling.

Why not use many more cells?

More cells can represent smaller geographic regions and reduce **quantization error**, the error introduced by replacing a continuous location with a representative center. But more classes generally leave fewer examples per class and create a harder classification problem. The choice is a resolution-versus-data trade-off.

Worked example: two cameras are 8 km apart but are predicted into the same cell. Both outputs use the same centroid. Even if both cell predictions are correct, at least one camera need not be close to that center. Correct class and exact coordinate are different achievements.

Name	Input	Purpose in this project
K-means, K = 300	Training coordinates	Construct geographic classes
KNN, k = 5	ResNet image-feature vectors	Retrieve similar training photographs for L7

Checkpoint: Which of these algorithms clusters GPS coordinates, and which searches image features? Answer key: A3.

4. From an image file to a tensor

Pixels, channels and tensors

A **pixel** is one position in an image. **RGB** represents its red, green and blue components. A **tensor** is a multidimensional numeric array. The network receives a tensor, not a JPEG file or a map.

The evaluation pipeline converts the image to RGB, resizes its shorter side to 256 pixels while preserving aspect ratio, then takes a central 224-by-224 crop. Resizing makes input scale consistent; center cropping produces the fixed dimensions expected by this pipeline. Cropping can also remove useful clues at the edges.

The RGB values are converted to floating-point values in [0, 1]. Each channel is then normalized using the ImageNet mean and standard deviation: red (0.485, 0.229), green (0.456, 0.224), blue (0.406, 0.225).

Normalization here means subtracting a channel mean and dividing by its standard deviation. It matches the input convention of the pretrained network. It is not probability calibration and does not remove every lighting variation.

Worked example: a red-channel value of 128 becomes approximately 128/255 = 0.502. Its normalized value is (0.502 - 0.485)/0.229 = 0.074. The operation changes the numeric representation, not the photograph's geographic label.

Tensor shapes through one prediction

Stage	Shape	Meaning
Prepared input	1 x 3 x 224 x 224	One image, three channels, height and width
Backbone output	1 x 2048 x 1 x 1	Pooled feature channels
Flattened descriptor	1 x 2048	One vector of image features
Classifier output	1 x 300	One score for each geographic class

Flattening rearranges the pooled values into a vector; it does not learn a new transformation. A **batch** contains several images processed together; its size replaces the first dimension above.

Training transformations are different

Earlier training uses random resized crops with scale 0.8-1.0 and random horizontal flips. Such **data augmentation** varies the appearance presented during training. L6 instead caches deterministic evaluation-style features. This difference is one reason the L6 gain cannot be attributed to geographic smoothing alone with perfect isolation.

The demo now uses lossless PNG transport for gallery image outputs. This avoids changing source RGB pixels through an extra lossy preview encoding before inference. Display size is separate from the network's 224-pixel crop.

Checkpoint: What is the difference between the displayed photograph and the tensor passed into ResNet? Answer key: A4.

5. What ResNet-50 actually does

A feature extractor learns useful representations

A **neural network** applies layers of learned mathematical transformations. Its learned numbers are **parameters**, commonly called weights and biases. A **feature** is a numeric representation that can help distinguish inputs. A **feature vector** collects those numbers for an image.

ResNet-50 is a residual convolutional neural network. The name refers to its standard 50-layer architecture; it does not mean 50 geographic classes. KPlaceNet removes its original image-classification output and uses it as a backbone that produces 2,048 features.

A **backbone** is the main representation-learning network. A **head** is the final task-specific component that converts its representation into the required output. In KPlaceNet, the head is a linear layer producing 300 geographic scores.

Main building blocks

A **convolution** slides learned filters across local image regions. A **filter**, or kernel, responds to visual patterns; early layers often represent simpler local structure, while later layers combine information across larger regions. This is a general interpretation, not proof that a specific prediction used a particular tree or road sign.

An **activation function** adds nonlinearity. ResNet uses ReLU, which outputs zero for negative values and retains positive values. Without nonlinear transformations, stacking linear operations would still be one linear transformation.

Batch normalization normalizes intermediate activations using batch statistics during training and stored statistics during evaluation. **Pooling** summarizes spatial information. At the end of this backbone, global average pooling reduces each feature channel to one number.

What makes it a residual network?

A **residual connection**, or skip connection, adds a shortcut to a learned transformation. In a compatible block, output = F(x) + x: learn a change F(x), then add the original representation x. When dimensions change, a projection adapts the shortcut.

These paths help deep networks optimize by making it easier to propagate information and gradients. They do not guarantee geographic accuracy or prevent overfitting.

Transfer learning

Pretraining means learning useful weights on an earlier task. **Transfer learning** reuses them for a new task. The global project uses ImageNet initialization. L5 starts from the selected global backbone, then adapts it to regional geolocation. L6 reuses that already-adapted backbone.

Checkpoint: What is the backbone, what is the head, and why not learn every visual pattern from scratch? Answer key: A5.

6. Scores, probabilities and learning

The linear classification head

Let f be the 2,048-number image descriptor. For geographic class j, the head computes z[j] = W[j] dot f + b[j]. W[j] contains the learned feature weights for that class and b[j] is its bias. The output z[j] is a **logit**, an unrestricted numeric score.

The head contains 2,048 x 300 weights plus 300 biases: **614,700 parameters**. This is the number updated in L6, not the number of parameters in the entire ResNet.

Softmax turns logits into nonnegative scores that sum to one: p[j] = exp(z[j]) / sum(exp(z[r])). **Argmax** selects the index with the largest score. In the classifier-only path, its cell ID determines the output coordinate.

Worked example: three logits [2, 1, 0] produce softmax scores of approximately [0.665, 0.245, 0.090]. Argmax chooses the first class. The scores sum to one because the system must distribute preference among its available classes, not because it has proved that any of those classes is appropriate.

How weights are learned

A **loss function** measures disagreement between a prediction and its training target. **Cross-entropy** penalizes low probability assigned to the target class. For a hard label y, it is -log(p[y]); confidently selecting the wrong class creates a larger penalty.

Backpropagation computes how the loss changes with respect to weights, using the chain rule. These derivatives are **gradients**. An **optimizer** uses gradients to update the weights. This is the learning step; displaying a prediction is not a learning step.

The **learning rate** controls update size. **Weight decay** discourages large weights and is one form of **regularization**, a technique intended to improve behavior beyond memorizing the training samples. The optimizer used for L6 is AdamW, which maintains adaptive gradient statistics and applies decoupled weight decay.

Batch, epoch and inference

A **mini-batch** is a subset used for one update. An **epoch** is one pass through the training examples. Twelve epochs do not mean twelve images or twelve layers. A **hyperparameter** is a setting chosen outside gradient learning, such as learning rate, batch size or number of neighbors.

Inference is applying the saved model to a new input without updating its weights. Evaluation mode uses stored batch-normalization statistics and disables training-time behaviors such as dropout. The demo performs inference; uploading more photographs does not retrain it.

Checkpoint: What changes during training that does not change when you press Locate photo? Answer key: A6.

7. L5: how the regional model began

Understanding the version names

L5, L6 and L7 label project experiments. They are not ResNet layers, product editions or progressively larger neural architectures.

Stage	What changes	What stays relevant later
L5 regional adaptation	Adapts a global backbone and regional head	Produces the regional backbone and 300-cell map
L6 geographic smoothing	Updates only the classifier head	Keeps that backbone and those cells
L7 retrieval refinement	Adds a training-feature bank and score blend	Keeps L6 weights and cell-centroid decoding

Fine-tuning and freezing

Fine-tuning updates pretrained weights on the new task. **Freezing** keeps chosen weights fixed. L5 fine-tunes ResNet's final major stage, called layer4, and its regional head. Earlier backbone stages remain frozen. The name layer4 is a stage identifier, not the fourth individual convolution.

L5 starts from the selected global L2 model, uses regional cells, and trains for 10 epochs with batch size 16, learning rate 0.0003, image size 224 and seed 42. The historical run used an RTX 4050 and mixed-precision training.

Mixed precision uses lower precision for selected calculations to reduce memory use and accelerate supported operations, while retaining suitable precision for sensitive steps. It is a compute technique, not a different geographic model.

Why regional specialization can help

A global model has to distinguish locations across the world with limited training coverage. Regional adaptation concentrates examples and class resolution within a smaller area. It changes both the training task and the meaning of its 300 outputs.

The regional L5 official-test baseline is 41.18% within 25 km with median error 42.08 km. These are coarse localization results, despite the historical name 'pinpoint mode'. Regional restriction itself gives the model information about the likely input area, so regional and global scores are not directly comparable.

The global and regional class maps are not interchangeable. Class 196 in one map need not represent the same place as class 196 in the other. A model checkpoint must be paired with the class definitions it was trained to predict.

Checkpoint: Which weights changed in L5, and why must the regional cell file match the regional checkpoint? Answer key: A7.

8. L6: geographic label smoothing

Why hard class boundaries can be awkward

A **hard label** assigns all target probability to one class. Suppose a training camera lies near a class boundary. A nearby class may describe a geographically reasonable alternative, but ordinary hard-label training treats all incorrect classes as incorrect labels regardless of distance.

Label smoothing softens the target distribution. Uniform smoothing spreads some target mass across all classes. **Geographic smoothing** instead gives more of that mass to classes whose centroids are geographically near the training coordinate.

The implemented target

q = 0.9 x one_hot(y) + 0.1 x softmax(-d / 15 km)

Here y is the assigned class, **one_hot(y)** is a vector with one at class y and zero elsewhere, d contains distances from the training camera coordinate to every cell centroid, and q is the distribution used as the cross-entropy target.

The assigned class keeps at least 90% target mass and also receives its share of the geographic 10%. A cell farther away receives less of the geographic share. The 15-km scale controls how quickly that share falls with distance; it is not a 15-km accuracy guarantee.

Worked example: imagine that the geographic share for three classes is [0.06, 0.03, 0.01] and the first class is the label. The final target is [0.96, 0.03, 0.01], rather than [1, 0, 0]. This is an illustrative target, not a measured example from the dataset.

What was trained and selected

The L5 backbone and all cells were fixed. Deterministic 2,048-dimensional descriptors were computed once and **cached**, meaning stored for reuse. Only the head was optimized. This makes repeated head updates much cheaper than repeatedly running or retraining the backbone.

Two fixed recipes, uniform and geographic smoothing, started from the same original head. Both used AdamW, learning rate 0.0001, weight decay 0.01, batch size 256 and 12 epochs. Validation within-25-km accuracy selected the best result, with median error breaking ties; the unchanged head was also eligible.

The geographic recipe at epoch 12 won. A selection record was saved before the official-test evaluation. Validation improved from 64.6% to 70.9% within 25 km; official-test performance improved from 41.18% to 47.62%.

Interpretation: the observed gain supports this bounded update. It does not prove smoothing alone caused every improvement, because cached deterministic training views also differ from L5's stochastic augmentation. No new training images were added.

Checkpoint: How is geographic smoothing different from uniform smoothing and from confidence calibration? Answer key: A8.

9. L7: what nearest-neighbor retrieval means

A descriptor represents appearance numerically

A **descriptor** or **embedding** is the feature vector produced by the backbone. Similarity in this learned space can be more useful than comparing raw pixel values: the vector summarizes patterns learned from earlier training.

The L7 **feature bank** stores descriptors and cell labels for the 15,939 training images. It excludes validation and official-test images. It is a searchable memory, not another ResNet and not a database of the correct answers for the gallery photographs.

At inference, the same frozen backbone extracts the uploaded image's descriptor. **K-nearest neighbors (KNN)** retrieves the k most similar descriptors from the bank. Here k = 5. KNN searches image appearance; it does not search for the nearest GPS coordinate of the upload, because that coordinate is unknown.

Cosine similarity

Cosine similarity measures the alignment of two vectors: similarity(f, g) = (f dot g) / (norm(f) x norm(g)). The **dot product** multiplies corresponding components and adds them. A vector's **norm** measures its length. Dividing by length emphasizes direction rather than overall magnitude.

Worked example: [1, 0] and [2, 0] have similarity 1 despite different lengths. [1, 0] and [0, 1] have similarity 0. Real descriptors have 2,048 dimensions, so these two-dimensional vectors only illustrate the calculation.

Do not confuse this feature-vector normalization with the RGB channel normalization in Chapter 4. One prepares image inputs; the other defines the retrieval similarity measure.

Weighted votes over classes

The five cosine similarities are divided by 0.07 and passed through softmax. This creates five weights that sum to one. Each neighbor gives its weight to its saved cell label; neighbors with the same label have their votes added.

The 0.07 **similarity temperature** controls how concentrated the five votes become. It was fixed for this experiment. It is not the 15-km geographic training scale and is not a fitted probability-calibration temperature.

The feature bank takes roughly 125 MiB for its descriptor values, plus labels and identities. Retrieval adds a similarity computation and memory use, but no new backbone training. The bank records its associated checkpoint hash; loading an incompatible bank is rejected.

Checkpoint: Where do neighbor labels come from, and why does a nearest neighbor not prove the uploaded photo is from this region? Answer key: A9.

10. Combining the classifier and KNN

A blend of two sources of evidence

The final distribution is p_final = 0.8 x p_classifier + 0.2 x p_neighbors. The classifier generalizes through learned weights; the neighbor branch provides local evidence from similar training appearances. Both branches use the same ResNet descriptor and the same geographic classes, so they are not independent models in a statistical sense.

The highest-scoring final class supplies the saved centroid. We combine class scores rather than averaging latitude-longitude coordinates, which could place a result between unrelated locations. The blend can change the winning class, but it does not create a new coordinate within the winning cell.

Worked example: retrieval changes the winner

For teaching purposes, replace the real 300 classes with just A, B and C. Suppose five neighbors have labels [A, B, B, C, C] and weights [0.30, 0.25, 0.20, 0.15, 0.10]. Their class votes are A = 0.30, B = 0.45, C = 0.25.

Class	Classifier score	Neighbor vote	80/20 combined score
A	0.42	0.30	0.8 x 0.42 + 0.2 x 0.30 = 0.396
B	0.40	0.45	0.8 x 0.40 + 0.2 x 0.45 = 0.410
C	0.18	0.25	0.8 x 0.18 + 0.2 x 0.25 = 0.194

The classifier alone chooses A. The hybrid chooses B. Its 0.410 score is relative combined support among these classes, not a 41% measured chance of being geographically correct.

Why these settings, and when can they hurt?

Five neighbors, temperature 0.07 and 20% retrieval weight were fixed as one conservative follow-up recipe, not established as optimal values. A neighbor branch can rescue a diffuse classifier prediction, but it can also reinforce misleading appearance matches. Similar vegetation or road textures do not establish a location.

The fixed hybrid passed the validation gate: within 25 km increased from 70.9% to 74.4%, and median error fell from 4.78 to 4.02 km. Test reporting followed that gate. KNN alone was also recorded as a diagnostic; it was not substituted after looking at its test score.

The interface checkbox switches between this hybrid and L6 alone. It applies to regional inference, not World mode. Selecting it does not alter saved weights. If the matching bank is absent, the app warns that the result uses ResNet alone.

Checkpoint: In the table, why does B win, and why does that not establish calibrated confidence? Answer key: A10.

11. Reading the measured results

Location error and accuracy at a radius

Ground truth is the labeled reference location. **Geodesic error** is distance over Earth's surface between the predicted coordinate and that reference. The implementation uses the haversine great-circle calculation, not driving distance or distance along roads.

Within-25-km accuracy is the fraction of labeled evaluation photographs whose error is at most 25 km. **Cell top-1 accuracy** is the fraction with the correct highest-scoring class. These metrics differ: an adjacent wrong class can still be close, and a correct large cell can have a centroid far from the camera.

Mean error is the arithmetic average and is sensitive to large failures. **Median error** is the middle error: roughly half the sample is at or below it. **P90 error** is the 90th percentile and describes the upper error tail. None is a confidence estimate for the next upload.

Measured result: the same 357 official test images

Metric	L5 baseline	L6 classifier	L7 hybrid
Within 25 km	41.18%	47.62%	53.50%
Images within 25 km	147 / 357	170 / 357	191 / 357
Median error	42.08 km	28.41 km	21.84 km
Mean error	102.85 km	85.33 km	80.65 km
Cell top-1 accuracy	17.93%	20.73%	25.21%
Within 1 km	0.56%	0.56%	0.56%

The L7 improvement over L6 is 5.88 **percentage points**, not 5.88% relative improvement: 53.50 - 47.62 = 5.88. Twenty-one additional images fall within 25 km. L5-to-L7 adds 44 such images.

The median and mean improve, but L7 p90 worsens from L6's 230.99 km to 239.82 km. Improvement is therefore uneven. Only two images are within 1 km in all three conditions; the model has not become an exact-address locator.

What the comparison supports

It supports an observed improvement on this bounded sample. The L7 official test was already inspected for L6, so L7 is a **reused-benchmark follow-up**, not a fresh independent confirmation. It is one fixed retrieval recipe with one backbone/seed. KNN alone records 53.78% within 25 km and median 18.90 km, but those diagnostic results do not justify choosing a new final method after seeing the test.

Checkpoint: How can median improve while p90 becomes worse? Answer key: A11.

12. Generalization, leakage and overfitting

Why keep three partitions?

Training examples fit weights or populate the KNN memory. Validation examples choose settings or decide whether to adopt a candidate. Test examples estimate performance after those choices. **Generalization** means performing usefully beyond the examples and conditions used for fitting and selection.

Data leakage occurs when information that should be unavailable influences training or model selection. Putting test photographs into the retrieval bank would be a direct leak. Repeatedly choosing settings by test performance also weakens the test's role as an independent check.

Overfitting occurs when a method fits the training data more closely than the broader task warrants. A train/test gap can be consistent with overfitting, but spatial differences and domain shift can also contribute. An improved test score does not establish that overfitting is solved.

Unique IDs do not guarantee independent scenes

The recovered train, validation and test image IDs are disjoint. But neighboring street photographs can show the same roads, buildings and vegetation despite having different file identities. This is **spatial dependence**.

The recovered-subset audit found that 982 of the 1,000 random validation coordinates are within 1 km of a training coordinate. Their median nearest-training distance is about 99 meters. None of the 357 official test coordinates is within 1 km of the recovered training set; the nearest is 1.005 km and the median is 2.520 km.

This helps explain why hybrid validation within-25-km accuracy is 74.4% while test accuracy is 53.5%. The random validation task can be easier because many views have nearby training examples. This audit covers the recovered regional subset, not every photograph in the original dataset.

Are the gallery examples cheating?

All ten gallery photographs were recovered from official test archives. The five regional examples are excluded from the training-only bank. Clicking a thumbnail loads pixels and scope; clicking Locate photo runs the model. The caption is not used as an answer lookup.

However, the examples were deliberately selected. A **curated example** illustrates a behavior; it is not a representative random sample. Showing four selected successes does not demonstrate 80% accuracy. The benchmark table and the failure example are needed for an honest account.

What stronger evidence would require

Use geographic/sequence groups and distance buffers to define validation before fitting the representation. Preserve a fresh final holdout, review duplicates, and repeat meaningful comparisons. Merely moving previously trained-on images into a new test folder does not make them unseen.

Checkpoint: Why can KNN score very well on random validation without that proving robust generalization? Answer key: A12.

13. Understanding the website's score

A score is not a measured probability of correctness

A **model score** expresses relative preference among available classes. A **calibrated probability** has been checked and adjusted so confidence levels correspond appropriately to observed outcomes on suitable data. Softmax alone does not guarantee this property.

With KNN off, the regional website displays the classifier's maximum softmax as a raw score. With KNN on, it displays the maximum combined retrieval score. Both are uncalibrated. Neither directly measures the chance of being within 25 km or the chance that the photograph was taken in Kerala.

Worked example: if a method displays 80% confidence on many images, a calibration assessment asks how often the corresponding predictions are correct. It does not assume the answer is 80%. Defining 'correct' also matters: exact cell and within-25-km correctness are different events.

Three scales that must not be confused

Setting	Used where	What it controls
15 km	L6 target construction	Geographic spread of the soft training target
0.07	L7 neighbor weighting	Concentration of the five similarity-based votes
Fitted temperature T	Global uncertainty workflow	Rescaling logits for confidence calibration

Why the Great Wall can get a confident regional answer

The classifier has a **closed set** of available geographic classes: there is no 'outside this region' output class. KNN likewise retrieves its nearest available matches even if all are poor matches. A score distribution can have a strong winner within the wrong set of choices.

Out-of-distribution (OOD) means the input differs materially from the conditions represented by the training/evaluation domain. A Chinese landmark is geographically out of domain for this regional task; an aerial view of Kerala may be geographically in-area but visually out of the street-view domain.

A separate rejection detector would need independent outside-region examples and a validated rule. A small score or low nearest-neighbor similarity might be explored as a signal, but no reliable OOD detector was established here. Clamping a predicted point inside Kerala would hide an error, not identify the real location.

Checkpoint: What should you say if an unfamiliar image gets a high combined score? Answer key: A13.

14. Trace one live prediction end to end

The complete regional inference path

1. Select a regional example or upload an image. The frontend provides image pixels and the selected scope. A gallery caption is for the person using the interface.

2. Convert the image to RGB, resize, center crop to 224 and normalize its channels. Move the tensor to the selected GPU or CPU device.

3. Apply the saved ResNet backbone in evaluation mode. Flatten its output into a 2,048-number descriptor.

4. Apply the saved 300-output linear head and softmax to obtain classifier scores.

5. If regional KNN is enabled and its matching bank is installed, retrieve five training descriptors using cosine similarity. Weight their class votes and combine them 80/20 with the classifier.

6. Choose the largest final class score, read its centroid and bounds from the saved cell map, and render the marker, coordinate and score.

7. If Open in Google Maps is selected, send that same coordinate in a Google Maps URL. Maps does not run the classifier or correct the prediction.

What the interface does and does not change

Changing the image, scope or KNN switch clears the previous result. Press Locate photo again after the new selection has loaded. Switching only the gallery tab does not change model scope by itself; choosing a photograph selects its associated scope.

Panning or zooming the map changes the view, not the prediction. Opening Prediction details exposes the recorded pipeline and class bounds. The bounds are geographic extents, not a confidence region. Repeatedly predicting the same prepared pixels with the same configuration is inference, not online learning.

Files and resources behind the screen

A **checkpoint** stores learned weights and associated model metadata. The regional L6 checkpoint is selected.pt; cells.json supplies the class geometry. The L7 train_bank.pt supplies training descriptors and labels. A **hash** is a file fingerprint used here to detect incompatible model/bank pairings; it does not measure model accuracy.

The checkpoint and bank are cached in application memory after loading. A **cache** avoids repeating expensive preparation, but is not a table of final answers for the gallery. GPU means graphics processing unit; CPU means central processing unit. The launcher uses CUDA, NVIDIA's GPU-compute platform, when available and otherwise uses CPU.

Checkpoint: Which steps run when a photo is selected, and which require Locate photo? Answer key: A14.

15. World mode and uncertainty

Keep these ideas separate from the Kerala demo

World mode uses a separate global checkpoint and global class map. The regional KNN checkbox does not refine its output. The live global model uses historical temperature scaling, but its calibration artifact lacks a checkpoint identity hash; the interface explicitly says that identity is unverified.

Temperature scaling divides logits by a fitted positive T before softmax. Larger T generally makes scores less concentrated. Division by a positive number preserves score ordering, so this operation does not change the highest-scoring class or improve its centroid error by itself.

Abstention is a policy that declines to accept a prediction under a rule. The live global demo abstains below a 50% score threshold. It still displays the candidate location; abstention is not proof that the candidate is wrong. Regional mode has no calibrated abstention policy.

Conformal prediction: a set, not one certain answer

A **prediction set** contains several candidate classes. **Coverage** is the frequency with which the true class is included. A large set can have high coverage while being geographically unhelpful. Coverage is not top-1 accuracy.

For split-conformal classification, a calibration example's **nonconformity score** is s = 1 - p_true: lower probability for the correct class means a larger score. Let n be the number of separate conformal-calibration examples and alpha the desired miscoverage. Compute k = ceil((n + 1) x (1 - alpha)), then use the kth smallest score as the cutoff. **Ceil** rounds upward.

If k exceeds n, the conservative cutoff is infinity. Include classes satisfying 1 - p_class <= cutoff. The statistical guarantee relies on **exchangeability**, roughly the assumption that calibration and future examples can be treated symmetrically as draws from the same process. It is marginal, not a guarantee for every town or shifted domain.

Worked example: scores [0.1, 0.2, 0.4, 0.8], n = 4 and alpha = 0.2 give k = ceil(5 x 0.8) = 4, cutoff 0.8. With alpha = 0.1, k = 5, beyond n, so use infinity. An interpolated percentile is not the same finite-sample rule.

The corrected global experiment is separate

Its 20,000-image model study separates 500 rows for temperature fitting, 1,000 for conformal calibration and 1,500 for evaluation. Expected calibration error (ECE), a summary of confidence-versus-accuracy mismatch across groups, falls from 0.4171 to 0.0453. Coverage is 90.8%, but the mean set has 159.7 of 300 classes. These figures belong to that recorded experiment, not the legacy live global model or the Kerala hybrid.

Checkpoint: Why must you not say 'our model is 90.8% accurate'? Answer key: A15.

16. Papers, new data and future improvements

PlaNet: the original classification idea

PlaNet, by Weyand and colleagues, formulates photo geolocation as classification over geographic cells using a convolutional network. KPlaceNet follows that broad approach, but uses different architecture, cell construction, data scale and experimental conditions. This is inspiration, not reproduction of PlaNet's published benchmark.

The confirmed ETH-related paper

Lindenberger and colleagues' Scaling Image Geo-Localization to Continent Level combines learned ground-view prototypes, aerial embeddings and geographic interpolation. A **prototype** is a learned representative vector. **Aerial imagery** views the area from above; **cross-view learning** relates representations across different viewpoints.

Its **retrieval** searches representations to identify locations. Our training-image KNN is also retrieval in the broad sense, but is not the same algorithm, representation or system. L6 adopts a broad geographic-boundary-awareness idea through soft labels; it does not implement that paper's aerial encoder, S2 hierarchy, prototype interpolation or multi-similarity objective.

Do not borrow the paper's accuracy figures for KPlaceNet or describe a broad conceptual connection as a reproduced method. Its contribution is relevant related work and a possible direction for a larger future system.

What the additional-data search actually produced

The search collected 350 Wikimedia Commons candidate metadata records around seven Kerala cities. A **candidate record** identifies an image and its reported coordinates, source and license fields. It is not automatically a suitable, correctly labeled training image.

No Commons image was added to L6 or L7 training, selection or test reporting. Actual scene suitability, camera coordinates, licenses, visual duplicates and capture sequences still need review. The Munnar query was unavailable during collection. The candidates are future-data leads, not an explanation for the measured gain.

The next defensible improvement

First define geographic/sequence holdouts, review new imagery and remove duplicates. Then compare methods under that fixed protocol with suitable baselines. A **baseline** is a reference method used to judge whether a change helps. Repeat runs where useful, report tail errors as well as medians, and preserve a fresh final test.

Use a state polygon if the intended task is strictly Kerala administration, and independently evaluate outside-region rejection if that is a requirement. These solve different problems: restricting the class map does not itself detect an input from outside that map.

Checkpoint: Did additional Commons training data or the complete ETH model produce today's improvement? Answer key: A16.

17. Code, artifacts and contribution

A practical code-reading route

Paths below are relative to work/KPlaceNet. Read them in this order to connect concepts to implementation; do not try to memorize the entire repository.

File or artifact	What to look for
src/dataset.py	RGB loading, get_transforms, training/evaluation transformations
src/model.py	GeoClassifier, backbone, head and forward pass
src/cells.py	Building cells and assigning geographic labels
src/retrieval.py	Cosine similarity, top five neighbors and class-vote accumulation
demo/app.py	Regional prediction, optional blend, frontend callbacks and Maps link
scripts/run_l6_kerala_head.py	Geographic targets, head updates, validation selection before test
scripts/evaluate_l7_knn.py	Training-only bank use, fixed blend and validation gate
src/uncertainty.py	Finite-sample conformal rank and prediction sets

The L6 and L7 folders under docs/results contain protocols, selection records, measured results and deployment manifests. A **protocol** specifies the experiment and selection rules. **Provenance** records where data and artifacts came from. A **manifest** lists files, identities or fingerprints needed to reproduce/check a run.

Checkpoints are separate from source code

A Git repository stores code and the small evidence files committed here. Model releases supply the large weights and retrieval bank. Pulling code alone does not install those assets. The install script verifies SHA-256 file fingerprints; the model also checks that its bank belongs to the same checkpoint.

On this laptop the installed regional weights are checkpoints/l6_kerala_geographic/selected.pt, the cell map is the adjacent cells.json, and the bank is checkpoints/l7_kerala_hybrid/train_bank.pt. Dataset images used for experiments are separate again; the full training image collection is not needed just to run an already-installed bank.

Attribute the work accurately

The existing project supplied the original global/regional pipeline and historical experiments. Prajwal's documented contributions include conformal-rank correctness, separation of fitting/evaluation roles, provenance checks, controlled follow-up studies, frontend integration, L6 geographic smoothing and the L7 retrieval experiment. These are project contributions; ResNet, K-means, KNN and the published methods were not invented here.

Repository integration is an administrative matter, not a model result. The fork branches were pushed and PR #1 was updated; the original Asterisk-Hunter main needs a maintainer merge unless that status has subsequently changed. Do not claim both mains are synchronized without checking.

Checkpoint: Which file proves the measured result, and why is a website screenshot insufficient as benchmark evidence? Answer key: A17.

18. Practice explaining, not reciting

Three levels of explanation

Twenty seconds: describe the input, ResNet features, geographic classes, optional neighbor refinement and centroid output. Do not start with the optimizer or a list of percentages.

Two minutes: add how labels are constructed, what changes in L6/L7, why the test split matters and what the displayed score does not mean. Use the toy hybrid calculation if someone asks how KNN changes the result.

Five minutes: follow the Demo Playbook. Explain one successful regional example, compare the KNN switch, show a failure, then connect those observations to the full benchmark. World mode is optional.

Questions an examiner may ask

1. Why use classification instead of directly predicting coordinates? What precision limit follows?
2. What are the model's input and training target? Where do GPS coordinates enter the procedure?
3. What does a convolution learn? What is a residual connection? What is transferred from pretraining?
4. Which weights change in L5 and L6? What changes in L7 if no weights are trained?
5. Why does a nearest neighbor need a ResNet descriptor? Why not match raw pixels?
6. What exactly do the 80% and 20% multiply? Are we averaging two coordinates?
7. Why can a wrong class be within 25 km? Why can a correct class have distance error?
8. Why are validation results higher? Can the model be overfitting even with disjoint image IDs?
9. Can a high score identify an outside-region image? Why did the Great Wall get a regional answer?
10. What changed after inspecting the test? Which reported result is a reused-benchmark follow-up?
11. Why is KNN-only not the deployed choice even though its diagnostic score is slightly higher?
12. Which methods did the group implement or adapt, and which are published methods being reused?

A useful rehearsal method

Answer each question without looking. Then check the relevant chapter and correct the explanation in your own words. If a term appears in your answer, be able to define it. Do not replace uncertainty with a confident claim about a model behavior that was never measured.

Readiness check: you should be able to draw the pipeline, compute the three-class blend, explain the metric table and recover from a failed upload. Reading this guide once is not the same as demonstrating those skills.

19. Chapter checkpoint answers

A1-A6: foundations

A1: the network selects a geographic class and displays its stored center. Decimal precision in that center does not create finer learned resolution.

A2: no. Regional mode chooses the regional weights and classes under an input-area assumption. Its rectangle includes neighboring territory and it has no verified outside-region detector.

A3: K-means clusters training GPS coordinates into 300 classes; KNN retrieves five similar training-image descriptors for the final regional refinement.

A4: the display is a human preview. The model receives an RGB-derived, normalized tensor with fixed 224-by-224 spatial dimensions; display resizing and the network's crop are separate.

A5: the backbone extracts the representation; the head maps it to 300 scores. Transfer learning reuses useful learned visual structure, reducing the data and compute needed compared with learning from scratch.

A6: training updates weights using loss gradients and an optimizer. Locate photo runs saved weights; it does not update them or add the upload to the retrieval bank.

A7-A11: the final methods

A7: L5 updates layer4 and its head. A matching cell map is essential because each head output was trained to represent one particular class definition.

A8: uniform smoothing spreads mass across all classes; geographic smoothing favors nearby centroids using training GPS. Confidence calibration adjusts score reliability on held-out data and is a different operation.

A9: labels come from training GPS converted to classes. A nearest available vector always exists even when all matches are poor or the input is outside the regional task.

A10: B receives enough neighbor support for its combined score 0.410 to exceed A's 0.396. Neither a weighted vote nor softmax alone establishes calibration against real outcomes.

A11: many central errors can shrink while some large errors grow. Median and p90 summarize different parts of the error distribution; reporting both exposes that trade-off.

Checkpoint answers continued

A12-A17: evidence and interpretation

A12: nearby training/validation scenes can share appearance despite different IDs. The random validation split has many such neighbors, making it easier than a separated task.

A13: it is a strong preference within the available regional classes, not evidence that the photograph belongs to that region or that its coordinate is correct.

A14: selecting a photograph loads the pixels and associated scope and clears stale results. Locate photo runs preprocessing, feature extraction, scoring, optional retrieval and centroid display.

A15: 90.8% is candidate-set coverage in a separate global uncertainty experiment, with large sets. It is not the fraction of correct top locations in the Kerala demo.

A16: no. The original regional subset was restored; L6 changed the head and L7 added a training-only feature bank. Commons records are future candidates, and the complete ETH system was not reproduced.

A17: use saved results with protocol, sample identities and selection records. A screenshot shows one selected case and cannot establish representative average performance.

Optional reference: the global experiments

Why these results are secondary in the demo

The global studies explain the broader project, but they are different tasks and model instances from the current regional hybrid. A common evaluation radius for them is 200 km rather than 25 km. Their percentages cannot be compared directly with Kerala's narrower geographic task.

Baseline and transfer-learning comparisons

L1 uses an ImageNet-pretrained ResNet backbone kept frozen and trains a 300-class global head on 10,000 images. Its recorded test result is 2.53% within 200 km. It establishes a reference point rather than strong worldwide performance.

Historical L2 compares initialization, the fraction of the 10,000-image subset, and whether layer4 is fine-tuned. **Initialization** means the starting weights. **Places365** is scene-oriented pretraining; **ImageNet** is object-oriented pretraining. The selected ImageNet/full-subset/layer4 condition records 3.33% within 200 km. '100% data' in that study means all 10,000 selected images, not all OSV-5M.

Geographic cell construction

A **quadtree-style spatial partition** recursively divides geographic space; the implemented global baseline uses density-driven spatial subdivision. **DBSCAN** groups points by local density and can mark sparse points as noise; this project's adapted version reassigns/merges/splits to produce its target class count. K-means groups around centers as described earlier.

Historical L4 compares these cell choices under a bounded recipe: within-200-km results are 3.33% for the spatial-tree method, 4.07% for K-means and 3.47% for adapted DBSCAN. Different cell definitions require their own heads and metadata. These historical results do not replace the live global checkpoint automatically.

Controlled follow-up studies

Class imbalance means some classes have many more examples than others. Inverse-frequency sampling gives rarer classes more sampling weight. A three-seed follow-up found no consistent gain from this balancing under its fixed recipe. A negative result is useful evidence about the tested method, not a reason to hide the experiment.

A separate paired three-seed 10k-versus-20k study records within-200-km accuracy 3.756% to 5.300% and mean error 5,977.3 to 5,521.6 km. **Paired seeds** mean each comparison uses matched random seeds to reduce unrelated variation. More examples also produce more updates at equal epochs, so data scale and optimization count are not perfectly separated.

Use the individual reports for exact samples and configurations: l1_baseline.md, l2_data_efficiency.md, l4_adaptive_cells.md, l2_sampling_osv_test_20261008.md and l2_data_scale_osv_test_20261009.md under docs/results. Do not transfer a newer study's results to an older installed model.

Optional reference: the main equations

From pixels to one location

Classifier: f = backbone(image), z = Wf + b, p[j] = exp(z[j]) / sum(exp(z[r])). f is a 2,048-number descriptor; p has one score per class. The largest score chooses a class.

L7 retrieval: similarity(f, g) = (f dot g) / (norm(f) x norm(g)). Retrieve the five largest similarities, softmax-weight them after division by 0.07, and add weights that share a class label.

Hybrid: p_final = 0.8 x p_classifier + 0.2 x p_neighbors. Select argmax(p_final), then return that cell's stored latitude and longitude. The score combination does not average geographic coordinates.

Training target and loss

Hard-label cross-entropy: loss = -log(p[y]), where y is the assigned class. For a soft target q, loss = -sum(q[j] x log(p[j])). This lets nearby geographic classes receive partial target mass.

L6 target: q = 0.9 x one_hot(y) + 0.1 x softmax(-d/15). d is the vector of training-camera-to-centroid distances in kilometers. Neither test GPS nor uploaded GPS supplies this training target during inference.

Great-circle distance for evaluation

Earth's surface is curved, and a degree of longitude represents different distances at different latitudes. Raw Euclidean distance between latitude-longitude degree values is therefore not a consistent kilometer measure.

For two coordinates, convert angles to radians. Let d_lat and d_lon be their angular differences. Compute:

a = sin(d_lat/2)^2 + cos(lat1) x cos(lat2) x sin(d_lon/2)^2

c = 2 x atan2(sqrt(a), sqrt(1-a)); distance = R x c

R is the Earth-radius constant used by the implementation (approximately 6,371 km). This **haversine great-circle approximation** gives surface separation on a sphere. It is not a road route, altitude-aware path or ellipsoidal survey calculation.

A metric is an aggregate, not an upload score

Within-25-km accuracy = count(errors <= 25 km) / number of evaluated photographs. For the final hybrid this is 191/357, approximately 53.50%. A probability-like score for a single input cannot substitute for those labeled errors.

Evidence and reading references

Implementation and measured records

The primary source for what this particular project does is its code. The primary source for the numerical claims is its saved experiment evidence, not a published paper's results.

[1] L5 historical regional run: docs/results/l5_kerala_pinpoint.md.

[2] L6 method and evidence: docs/results/l6_kerala_20261020.md and its results, protocol, split_audit and selection_before_test JSON files. https://github.com/Prajwal-k-tech/KPlaceNet/blob/main/docs/results/l6_kerala_20261020.md

[3] L7 fixed retrieval follow-up: docs/results/l7_kerala_knn_20261020.md and associated protocol/results/selection JSON files. https://github.com/Prajwal-k-tech/KPlaceNet/blob/main/docs/results/l7_kerala_knn_20261020.md

[4] Separate global uncertainty study: docs/results/l3_uncertainty_osv_test_20261009.md. The 500/1,000/1,500 fitting/calibration/evaluation partitions and its results belong to that study.

[5] Global data-scale follow-up: docs/results/l2_data_scale_osv_test_20261009.md. Across three paired seeds, 10k versus 20k training images increased within-200-km accuracy from 3.756% to 5.300% and reduced mean error from 5,977.3 to 5,521.6 km. These are historical global comparisons, not Kerala or live-World figures.

Research sources already used by the project

He et al. Deep Residual Learning for Image Recognition. CVPR 2016. https://arxiv.org/abs/1512.03385

Weyand et al. PlaNet: Photo Geolocation with Convolutional Neural Networks. ECCV 2016. https://arxiv.org/abs/1602.05314

Astruc et al. OpenStreetView-5M: The Many Roads to Global Visual Geolocation. CVPR 2024. https://arxiv.org/abs/2404.18873 ; dataset: https://huggingface.co/datasets/osv5m/osv5m

Lindenberger et al. Scaling Image Geo-Localization to Continent Level. NeurIPS 2025. https://arxiv.org/abs/2510.26795 ; https://scaling-geoloc.github.io/

Guo et al. On Calibration of Modern Neural Networks. ICML 2017. https://proceedings.mlr.press/v70/guo17a.html

Angelopoulos and Bates. A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification. https://arxiv.org/abs/2107.07511

Additional-data candidate source: Wikimedia Commons API and machine-readable licensing metadata. https://commons.wikimedia.org/wiki/Commons:API/MediaWiki ; https://commons.wikimedia.org/wiki/Help:Machine-readable_data

Use these sources correctly: the papers explain methods and context. Only the local run records support KPlaceNet's reported measurements. The regional results do not establish performance on all Kerala photographs, and the reused L7 benchmark is not fresh confirmation.